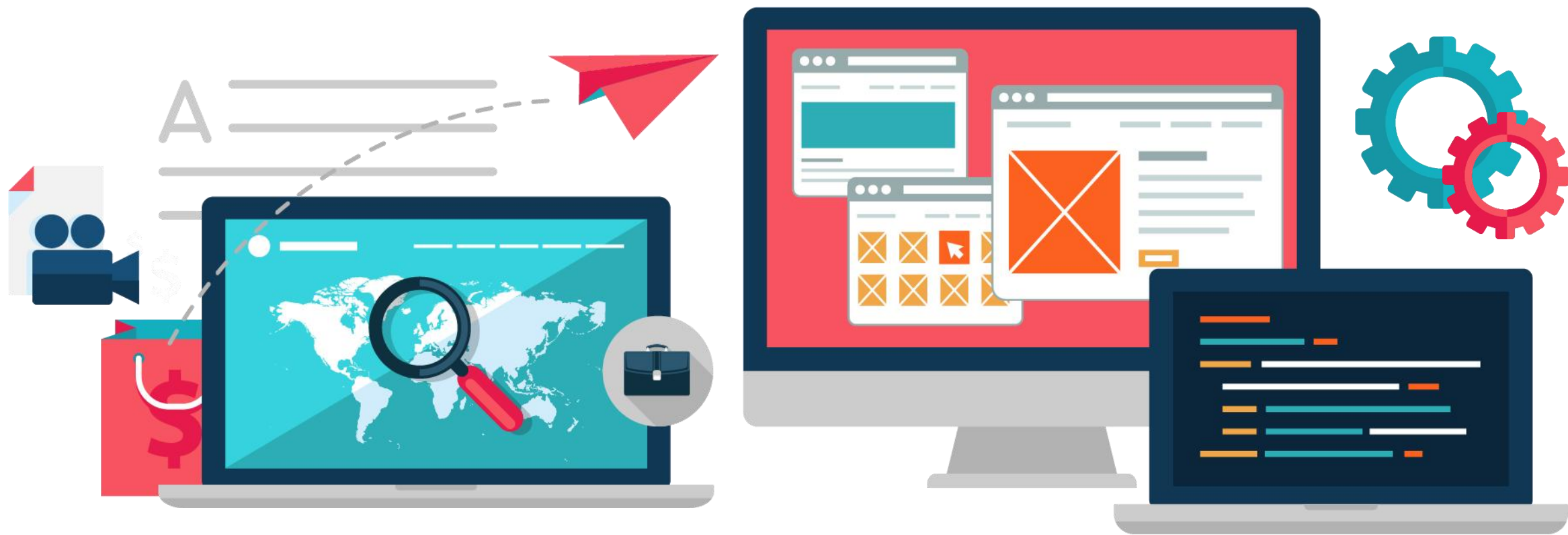


面向对象程序设计

Object Oriented Programming





C++ 程序结构



01

C++工程(项目)组成

02

最简单的C++程序

03

C++的多文件结构

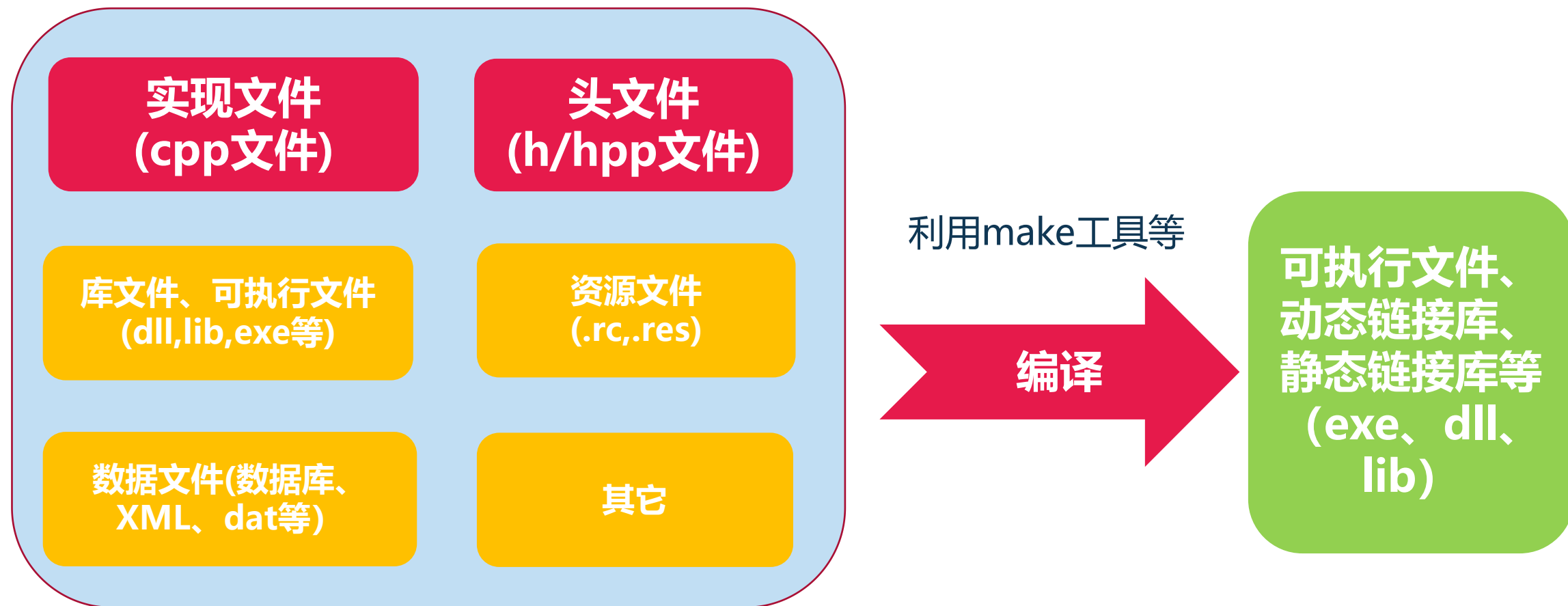
04

包含警戒

05

C++项目编译过程





◆单文件C++程序

```
/* helloworld.cpp */  
// my first program in C++  
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    cout << "Hello World!" << endl;  
    return 0;  
}
```

- 块注释
- 行注释
- 预编译命令#include
- using指令
- 主函数，用户代码的主要入口点
- 标准输出流对象cout
- << 插入/输出运算符
- endl 流操纵器，实际为函数
- return语句



- ◆ C/C++普通可执行程序(exe)的起始函数
- ◆ 返回值: int (只有早期的部分编译器支持void main())
- ◆ 参数部分: 无 或 (int argc, char* argv[])

//过时的, C89/90

```
//例: appMain.cpp
#include <iostream>
using namespace std;

void main( )
//或void main(int argc, char* argv[ ])
{
    //...
}
```

//标准的, C99和C++98

```
//例: appMain.cpp
#include <iostream>
using namespace std;

int main( )
//或int main(int argc, char* argv[ ])
{
    //...
    return 0;
}
```



//假设最终生成的exe文件为
c:\xyz\my.exe

```
#include <iostream>
```

```
using namespace std;
```

```
int main(int argc, char* argv[ ])
```

```
{
```

```
    cout<<"argc = "<<argc<<endl;
```

```
    for(int i=0;i<argc;++i) {
```

```
        cout<<"argv["<<i<<"]="";
```

```
        cout<<argv[i]<<endl;
```

```
    }
```

```
    return 0;
```

```
}
```

在my.exe所在的目录下,

执行:

my.exe回车

输出:

argc=1

argv[0]= my.exe

若执行:

my.exe 123 computer回车

输出:

argc = 3

argv[0] = my.exe

argv[1] = 123

argv[2] = computer



◆稍复杂的单文件C++程序

```
//area.cpp
#include <iostream>
using namespace std;
const float PI=3.14f; //声明常量
float area (float r)    //声明函数
{   return PI*r*r; }    //同时定义
int main( )
{
    cout << area(3) << endl;
    return 0;
}
```

```
//area.cpp
#include <iostream>
using namespace std;
const float PI=3.14f; //声明常量
float area (float r);  // 前置声明函数
int main( )
{
    cout << area(3) << endl;
    return 0;
}                                // 定义函数
float area (float r) {   return PI*r*r; }
```



- ◆ 小程序通常可以在单个程序文件中编写完成。然而，如果程序规模变得较大，或者需要多个开发人员共同完成，就有必要将程序拆分成多个程序文件，使代码更加易于组织管理。

//文件1, **area.cpp**

```
const float PI=3.14f;  
int gcount = 0;  
float area (float r) {  
    gcount ++;  
    return PI*r*r;  
}
```

//文件2, **main.cpp**

```
#include <iostream>  
using namespace std;  
extern int gcount;  
float area (float r);  
int main( ) {  
    cout << area(3) << gcount << endl;  
    return 0;  
}
```




◆调优的方案

//文件1, **area.h**

```
const float PI=3.14f;
```

```
extern int gcount;
```

```
float area (float r);
```

//文件2, **area.cpp**

```
#include "area.h"
```

```
int gcount = 0;
```

```
float area (float r) {
```

```
    gcount ++;
```

```
    return PI*r*r;
```

```
}
```

//文件3, **main.cpp**

```
#include <iostream>
```

```
using namespace std;
```

```
#include "area.h"
```

```
int main( ) {
```

```
    cout << area(3) << gcount << endl;
```

```
    return 0;
```

```
}
```



头文件是包含程序接口声明和共享定义的文本文件，通常包括：函数/类/变量声明、全局常量定义、类型定义、内联函数定义、模板定义和宏定义。

//例: dog.h

```
#ifndef DOGH
```

```
#define DOGH
```

```
#include <iostream> // C++标准库头文件
```

```
#include "headfile1.h" // 自定义头文件
```

```
#include "headfile2.h"
```

```
class Bone;
```

```
class Dog {
```

```
public:
```

```
    void Eat(Bone * pBone);
```

```
    //...略
```

```
};
```

```
#endif
```

① 头文件扩展名

② 标准库与自定义头文件

③ include的顺序

④ 前置声明

⑤ 使用头文件

⑥ 包含警戒(include guard)

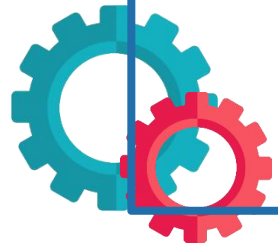
⑦ 包含警戒与#pragma once

- ◆ 标准库：包括头文件和库文件两部分，由编译器供应商提供实现，并随编译器一同发布。
- ◆ 属于标准C++的一部分。
- ◆ `#include <a.h>`和`#include "a.h"`
 - `<>`：在系统目录中查找相应的头文件
 - `"`：先在本地(当前工程)的目录中查找相应的头文件；若找不到，再到系统目录中查找相应的头文件
- ◆ `#include <xx>`和`#include <xx.h>`
 - ◆ 前者是C++的风格
 - ◆ 大体上：`#include <xx>`相当于

```
namespace std {  
    #include "xx.h"  
}
```

```
// C标准库头文件  
#include <stdlib.h>  
#include <math.h>  
// C兼容头文件  
#include <cstdlib>  
#include <cmath>  
// 自定义头文件  
#include "myfileX.h"
```

1. 全部以`#include "a.h"`形式定义好吗？
2. 使用`<>`和`"`时，哪个形式放前边？



前置声明：告诉编译器，一个标识符所代表的类型、含义等，但不必告知其具体位置

```
// myfile77.h
#ifndef MYFILE77H
#define MYFILE77H
struct A;
struct B;
void func1st( A * );
void func2nd( B * );

struct A { B *  pB; };

struct B { A * pA; };

#endif
```

```
//myfile77.cpp
#include "myfile77.h"

void func1st(A * pa) {
    ...
}

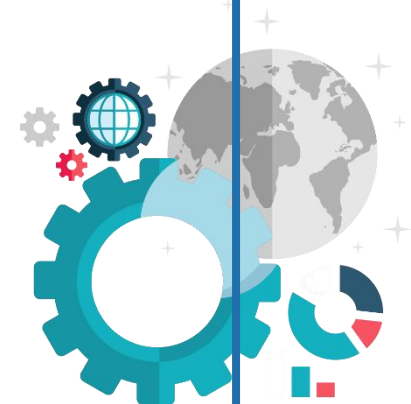
void func2nd(B * pb) {
    ...
}
```

有一个项目，由三个人共同完成，每个人都要使用其他人的全局变量、函数和结构。

my.cpp文件: 定义了全局变量 `int a=0;`
定义了全局自由函数 `void myFunc( );`
定义了全局结构 `struct A { /*略*/ };`

your.cpp文件: 定义了全局变量 `int b=99;`
定义了全局自由函数 `void youFunc( );`
定义了全局结构 `struct B { /*略*/ };`

his.cpp文件: 定义了全局变量 `int c=88;`
定义了全局自由函数 `void heFunc( );`
定义了全局结构 `struct C { /*略*/ };`





```
// 定义一个公共的头文件:  
ourcommon.h  
//ourcommon.h  
extern int a;  
extern int b;  
extern int c;  
void myFunc( );  
void yourFunc( );  
void hisFunc( );  
struct A { /*略*/};  
struct B { /*略*/};  
struct C { /*略*/};
```

谁维护
该文件
我?
你?
他?

```
//my.cpp  
#include "ourcommon.h"  
int a = 0;  
...  
  
//your.cpp  
#include "ourcommon.h"  
int b = 99;  
...  
  
//his.cpp  
#include "ourcommon.h"  
int c = 88;  
...
```

☺ 从各自的c/cpp文件出发, 将其向外部公开的变量和函数的声明、结构和类的定义等, 放入对应的头文件。

```
//my.h
#ifndef MYH
#define MYH
extern int a;
void myFunc( );
struct A { /*略*/};
#endif
```

```
//my.cpp
#include "my.h"
#include "your.h"
#include "his.h"
int a=0;
void myFunc( )
{ ... }
```

```
//your.h
#ifndef YOURH
#define YOURH
extern int b;
void yourFunc( );
struct B { /*略*/};
#endif YOURH
```

```
//your.cpp
#include "my.h"
#include "your.h"
#include "his.h"
int b=99;
void yourFunc( )
{ ... }
```

```
//my.h
#ifndef MYH
#define MYH
extern int a;
void myFunc( );
struct A { /*略*/};
#endif
```

```
//his.cpp
#include "my.h"
#include "your.h"
#include "his.h"
int c=88;
void hisFunc( )
{ ... }
```



包含警戒 (include guard) 的引入

```
//my.h  
extern int a;  
void myFunc( );  
struct A { /*略*/};
```

```
//my.cpp  
#include "my.h"  
#include "my.h"  
#include "your.h"  
#include "his.h"  
int a=0;  
void myFunc( ) { ... }
```

```
//your.h  
#include "my.h"  
extern int b;  
void yourFunc( );  
struct B { /*略*/};
```

```
//your.cpp  
#include "my.h"  
#include "your.h"  
#include "his.h"  
int b=99;  
void yourFunc( ) { ... }
```




用包含警戒 (include guard) 解决

```
//my.h
#ifndef MYH
#define MYH

extern int a;
void myFunc( );
struct A { /*略*/};

#endif
```

```
//my.cpp
#include "my.h"
#include "my.h"
#include "your.h"
#include "his.h"
int a=0;
void myFunc( ) { ... }
```

```
//your.h
#ifndef YOURH
#define YOURH
#include "my.h"
extern int b;
void yourFunc( );
struct B { /*略*/};
#endif
```

```
//your.cpp
#include "my.h"
#include "my.h"
#include "your.h"
#include "his.h"
int b=99;
void yourFunc( ) { ... }
```



//例: dog.h

```
#ifndef DOGH
#define DOGH
#include "headfile1.h"
class Bone;
class Dog {
public:
    void eat(Bone * pBone);
    //...略
private:
    int age;
    //...略
};
#endif
```

//例: dog.cpp

```
#include <iostream>
#include "dog.h"
#include "dog.h"
#include "bone.h"
void Dog::eat(Bone *pBone)
{
    std::cout<< "Eat one!" <<std::endl;
}
```

- ◆ 作用: 允许同一个cpp文件, include多次同一个头文件.
- ◆ 包含警戒的命名

- ◆ #pragma once与包含警戒功能上基本相同;
- ◆ 但部分编译器，特别是早期的，对#pragma once支持不好;

//例: dog.h

```
#ifndef DOGH
#define DOGH
```

```
#include "headfile1.h"
```

```
class Bone;
class Dog {
public:
    void Eat(Bone * pBone);
    //...略
private:
    int age;
    //...略
};
```

```
#endif
```

等价

//例: dog.h

```
#pragma once
```

```
#include "headfile1.h"
```

```
class Bone;
class Dog {
public:
    void Eat(Bone * pBone);
    //...略
private:
    int age;
    //...略
};
```

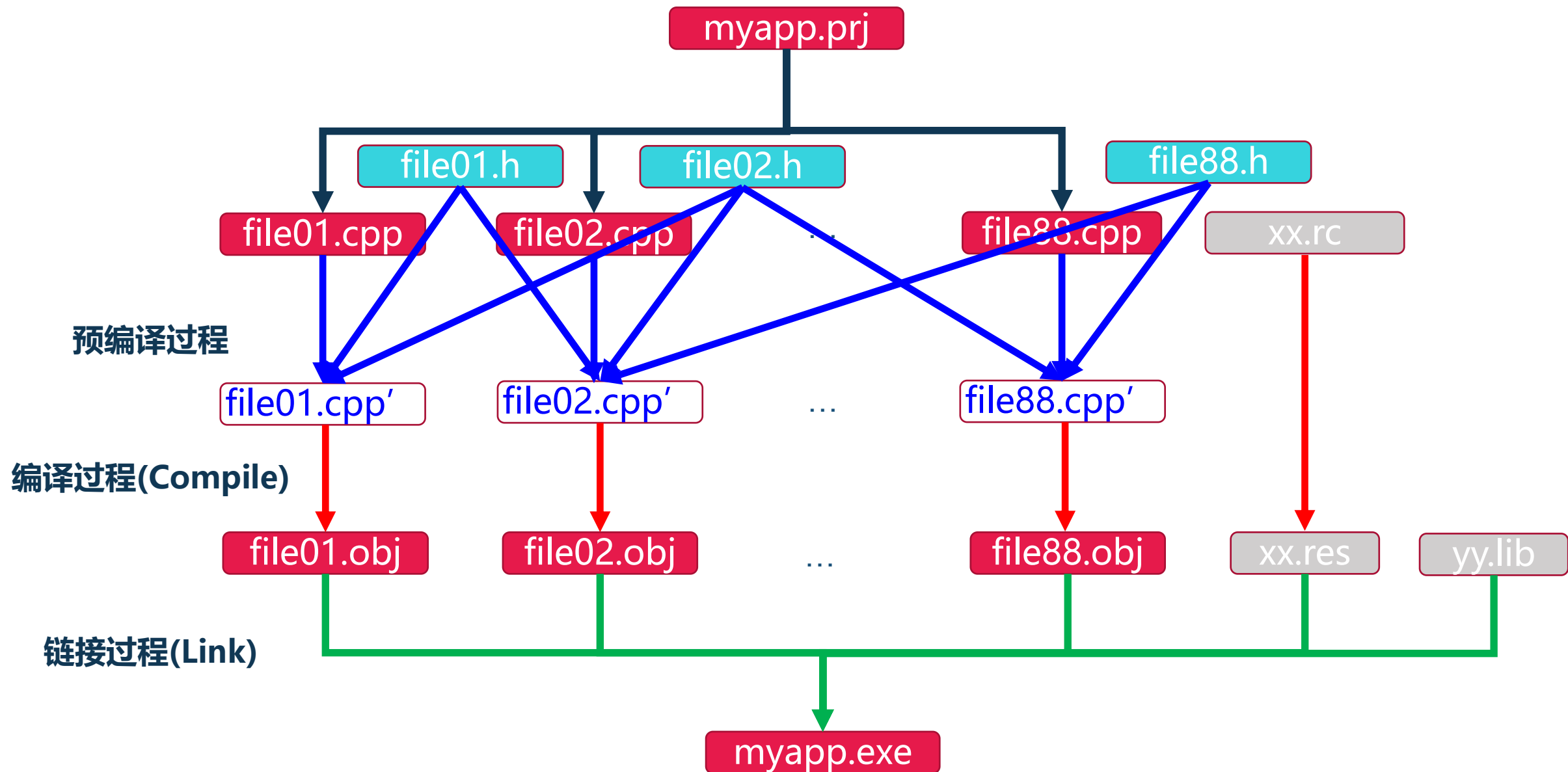
实现文件是包含定义的源代码文件，通常包括全局变量的定义、全局函数的定义(包括main函数)、类成员函数的定义、类静态成员的定义等，注意，不要#include “*.cpp”。

```
//例： appMain.cpp
#include <iostream>
using namespace std;

int main( )
{
    int nValue;
    cout<<“请输入整数:” ;
    cin>>nValue;
    cout<<“你输入的数为”
        <<nValue<<endl;
    return 0;
}
```

```
//例： dog.cpp
#include “dog.h”
#include <iostream>
#include “bone.h”
void Dog::Eat(Bone *pBone)
{
    std::cout <<“Eat bone!”<<std::endl;
}
```

```
//例： appMain.cpp
#include <iostream>
using namespace std;
#include “dog.h”
#include “bone.h”
int main( ) {
    Dog aDog;
    Bone aBone;
    aDog.Eat(&aBone);
    cout<<“Over.”<<endl;
    return 0;
}
```



预处理：预处理源代码中的带有#号的语句，生成编译程序可处理的文本文件；

```
//appmain.cpp  
#include "my.h"  
int main( ) {  
    f(2);  
    return 0;  
}
```

```
//my.h  
  
#define AAA  
#ifndef AAA  
    #include <string>  
#else  
    void f(int );  
#endif;
```

预编译

//预编译结果

```
void f(int );  
int main( ) {  
    f(2);  
    return 0;  
}
```



编译过程：对每个预处理后源程序，编译并生成相应的二进制目标文件(object文件)。
此过程中，**要求编译器能够识别每个标识符，知道其各自的类型、含义，但不要求知道存放位置。**

```
//appmain.cpp
#include "my.h"
int main( ) {
    f(2);
    return 0;
}
```

```
//my.h
#define AAA
#ifndef AAA
    #include <string>
#else
    void f(int );
#endif;
```

预编译

```
//appmain.cpp
#include <string>(实际会继续展开)
int main( ) {
    f(2);
    return 0;
}
```

编译

语法错误：不认识的标志符f;

问题：若把main错写成mian，在编译阶段编译器会报语法错误吗？



链接过程：对在整个程序范围内，确定各标志符所代表的地址，如变量、函数入口，生成可执行文件。

此过程中，**要求编译器能够确定每个标识符所对应的含义或地址。**

1. C++的编译是以实现文件(cpp文件)为基本编译单位的；
2. 预编译、编译和链接过程，有时我们统称编译过程；
3. 各阶段均无错误，才能生成可执行文件；
4. 即使生成了可执行文件，不代表程序就正确；
5. 让编译过程通过，是相对简单的；
真正困难的是调试逻辑错误



◆ make文件内容:

myapp.exe link.exe myfile01.obj myfile02.obj myfile88.obj

myfile01.obj cc.exe myfile01.cpp myfile01.h myfile02.h

myfile02.obj cc.exe myfile02.cpp myfile01.h myfile02.h myfile88.h

...

myfile88.obj cc.exe myfile88.cpp myfile02.h myfile88.h

例如：**myfile02.obj** cc.exe myfile02.cpp myfile01.h myfile02.h myfile88.h时，

1. 先取得**myfile02.obj**文件的时间戳(最后修改时间)，
2. 与myfile02.cpp myfile01.h myfile02.h myfile88.h各文件的时间戳比较，
3. 若某个文件(如myfile02.h)比**myfile02.obj**文件新，说明myfile02.h有新的修改，则执行这行命令；
4. 否则略过这行命令，加快编译速度。



本章结束